

PHASE 2 · WEEKS 4–6 · MULTI-SOURCE RETRIEVAL

Problem Statement

Build a parallel multi-engine retrieval system — structured, semantic, and graph — synthesized by a DSPy-optimized module.

Reference project: github.com/waseemkhan606/PhaseDosProject

AI Engineering Masters Program

Topics This Project Must Demonstrate

Hybrid search / BM25 / RRF · Text-to-SQL · Neo4j GraphRAG · DSPy declarative prompting · parallel multi-engine retrieval · LLM-as-judge optimization.

Reference Project Summary

The reference build — a "Synthetic Cancel Culture Sandbox" — fires three retrieval engines in parallel for a given entity name: SQLite for structured stats, FAISS for semantic document search, and Neo4j for relationship/graph traversal. A DSPy ChainOfThought module, compiled with BootstrapFewShot against a teacher/student model pair, synthesizes all three contexts into a single structured verdict, graded by a rule-based judge metric.

The Assignment

Build your own three-engine parallel retrieval system over a domain you choose. You must query a **structured store** (SQL), a **semantic store** (FAISS, with hybrid BM25+dense retrieval), and a **graph store** (Neo4j) simultaneously for the same entity, then use a DSPy module — compiled with a real optimizer, not just a hand-written prompt — to synthesize one structured answer from all three.

Choose Your Domain — Pick One

- 1. Startup Due-Diligence Engine** — SQL: funding/financials; FAISS: news/press semantic search; Neo4j: founder/investor/competitor graph. Verdict: INVEST / PASS.
- 2. Sports Team Scout** — SQL: player stats; FAISS: scouting report chunks; Neo4j: team/transfer/rivalry graph. Verdict: SIGN / SKIP.
- 3. Restaurant Health & Reputation Auditor** — SQL: inspection scores; FAISS: review text; Neo4j: ownership/chain/supplier graph. Verdict: SAFE / RISK.
- 4. Job Candidate Screener (mock data only)** — SQL: resume metrics; FAISS: cover-letter/portfolio text; Neo4j: reference/referral network. Verdict: ADVANCE / REJECT.
- 5. Landlord/Property Risk Checker** — SQL: violation records; FAISS: tenant review text; Neo4j: ownership entity graph. Verdict: RENT / AVOID.
- 6. Open-Source Maintainer Health Check** — SQL: commit/release cadence; FAISS: issue/PR discussion text; Neo4j: contributor/dependency graph. Verdict: HEALTHY / AT-RISK.
- 7. Supplier Risk Dashboard** — SQL: delivery/defect metrics; FAISS: contract/complaint text; Neo4j: subsidiary/ownership graph. Verdict: APPROVE / FLAG.
- 8. Campus Club/Event Trust Score** — SQL: attendance/budget records; FAISS: feedback text; Neo4j: organizer/sponsor network. Verdict: ENDORSE / REVIEW.
- 9. Open topic** — student-proposed three-store domain, subject to instructor approval.

Required Deliverables

- Synthetic dataset generator that populates all three stores (SQLite, FAISS, Neo4j) — script must be runnable and reproducible, like `generate_sandbox.py`.
- Three independent, individually testable query functions — one per engine — each returning a labeled context string.
- A parallel retrieval function (`ThreadPoolExecutor` or `async`) that fires all three simultaneously and tolerates any single engine failing without crashing the whole request.
- A DSPy signature + module, compiled with an optimizer (`BootstrapFewShot` or equivalent) using a teacher/student model split.
- A rule-based judge metric that validates output structure and enforces at least one hard content constraint (banned words, required fields, score ranges).
- A minimal UI showing all three raw contexts side by side plus the synthesized verdict — transparency into the retrieval, not just the final answer.
- README documenting the architecture diagram, tech stack, and setup for all three databases.

Out of Scope for This Phase

LangGraph agent orchestration, tool-calling loops, and multi-agent supervisors belong to Phase 3. Production observability, caching, and fallback cascades belong to Phase 4. The DSPy synthesis step should be a single compiled module call, not a multi-step agent.

How This Will Be Evaluated

- Do all three engines return genuinely different information for the same entity (not duplicated content)?
- Does the system degrade gracefully if one engine is down (e.g. Neo4j not running)?
- Is there evidence the DSPy module was actually compiled/optimized (saved state file, before/after prompt comparison)?
- Does the judge metric actually reject bad outputs — show at least one example it correctly failed.